

3D MARKETPLACE ARCHITECTURE

Upwork-Grade Platform Blueprint & Implementation Roadmap

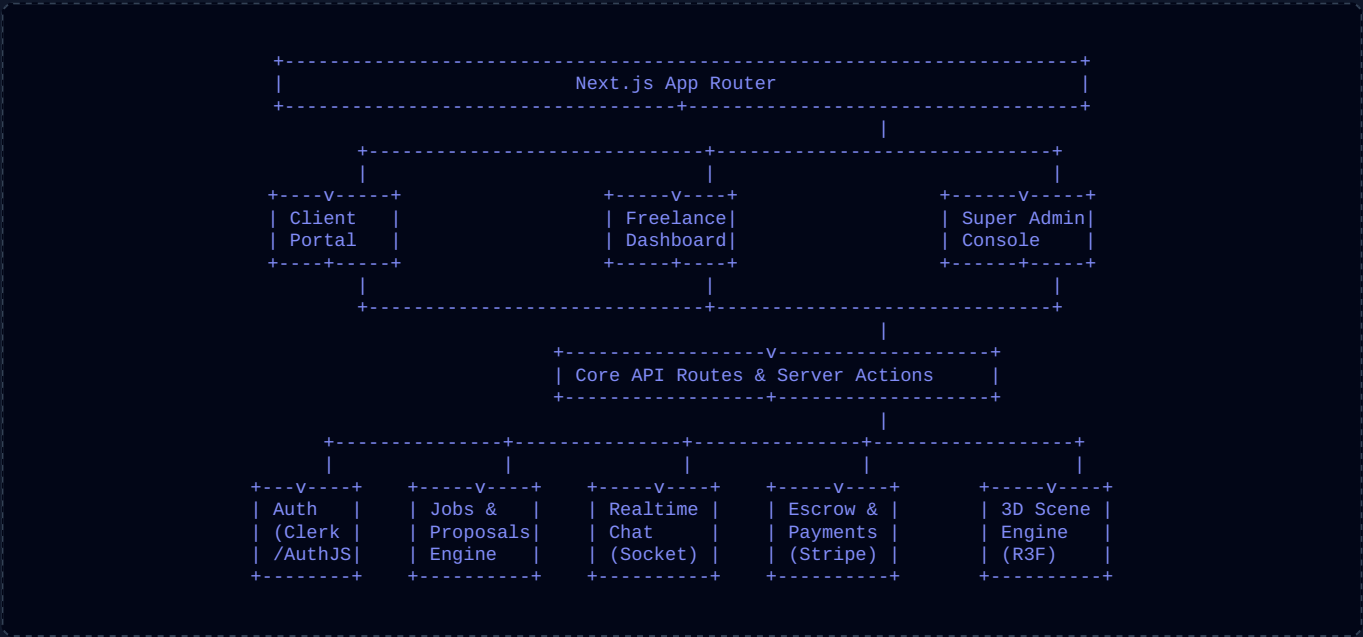
- Next.js 15 App Router
- React Three Fiber (R3F)
- Prisma ORM
- Stripe Connect Escrow
- PostgreSQL

Executive Summary

This document outlines the complete technical blueprint and end-to-end implementation plan for building a full-featured, scalable 3D Freelance Marketplace styled after Upwork. The system utilizes Next.js 15, dynamic 3D visual canvas rendering, real-time messaging, multi-tenant state management, and a secure escrow-based payment pipeline.

1. SYSTEM ARCHITECTURE & ROLE MODULES

The platform follows a multi-tenant role-based architecture dividing the user domain into three primary profiles: **Clients (Employers)**, **Freelancers (Talent)**, and **Super Admins**.



MODULE	FEATURES & WORKFLOW	KEY API ENDPOINTS
Identity & Access	Multi-role Auth (Client/Freelancer/Admin), Profile Switching, KYC Verification, Skill Taxonomy onboarding wizard.	/api/auth/* /api/user/profile
Job Engine	Job Posting Wizard, Category/Skill Tagging, Fixed vs. Hourly budget types, AI-assisted proposal matching.	/api/jobs/create /api/jobs/feed
Bidding System	Proposal Submission, Cover Letter, Custom Screening Questions, Milestone Breakdown, Bid Ranking.	/api/proposals/submit /api/proposals/[id]
Contract & Escrow	Offer Creation, Stripe Escrow Deposit, Milestone Funding, Work Submission, Fund Release, Dispute Management.	/api/contracts/offer /api/escrow/deposit

Module	Features & Workflow	Key API Endpoints
Realtime Engine	1-on-1 Direct Messaging, File Attachments, Dynamic Offer Cards, Real-time Notification System.	/api/chat/socket /api/notifications
Admin Console	Platform Metrics (GMV, Revenue), User Verification Overrides, Escrow Dispute Resolution, Fee Config.	/api/admin/users /api/admin/disputes

2. Recommended Tech Stack

Layer	Technology	Primary Purpose
Framework	Next.js 15 (App Router)	Full-stack React framework providing SSR, SSG, Edge routing, and Server Actions.
3D Rendering	@react-three/fiber + Drei	Interactive 3D dashboards, dynamic hero sections, and visual workspace node canvases.
Styling & UI	Tailwind CSS + Shadcn UI	Responsive layouts, modern glassmorphism aesthetics, and accessible interactive controls.
Database & ORM	PostgreSQL + Prisma ORM	Relational schema for strict ACID compliance on financial transactions and active contracts.
Realtime Engine	Supabase / Socket.io + Redis	Instant messaging, live proposal updates, and real-time online status indicators.
Payments & Escrow	Stripe Connect	Managed payouts, milestone escrow holds, multi-currency processing, and identity checks.

3. 3D UI/UX Integration Strategy

To preserve application performance while providing a visually stunning 3D user experience, 3D elements are strategically deployed on core key surfaces:

- **3D Interactive Hero (Landing Page):** Interactive floating 3D globe/workspace node mapping active clients and freelancers globally.
- **Interactive 3D Dashboard Canvas:** A 3D isometric overview showing project progress nodes (Pending, Active, In Review, Completed) connected by interactive glowing data paths.
- **Performance Optimization:**
 - Wrap all 3D Canvases in `next/dynamic` with `ssr: false` to prevent hydration mismatches.
 - Implement **React Suspense** with fallback skeletal loaders.
 - Utilize Draco compression (`.glb` / `.gltf`) to keep all 3D model assets under 2MB.

4. Database Schema Blueprint (Prisma ORM)

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

```

generator client {
  provider = "prisma-client-js"
}

enum Role {
  CLIENT
  FREELANCER
  ADMIN
}

enum JobType {
  FIXED_PRICE
  HOURLY
}

enum ContractStatus {
  PENDING
  ACTIVE
  IN_REVIEW
  COMPLETED
  DISPUTED
}

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  name        String
  role        Role        @default(FREELANCER)
  avatarUrl   String?
  bio         String?
  hourlyRate  Float?
  skills       String[]
  jobsPosted  Job[]       @relation("ClientJobs")
  proposals   Proposal[]
  contractsAsClient  Contract[] @relation("ClientContracts")
  contractsAsFreelance Contract[] @relation("FreelancerContracts")
  createdAt   DateTime    @default(now())
}

model Job {
  id          String      @id @default(cuid())
  title       String
  description  String
  type        JobType
  budget      Float
  skills       String[]
  clientId    String
  client      User        @relation("ClientJobs", fields: [clientId], references: [id])
  proposals   Proposal[]
  contracts   Contract[]
  createdAt   DateTime    @default(now())
}

model Proposal {
  id          String      @id @default(cuid())
  jobId       String
  job         Job         @relation(fields: [jobId], references: [id])
  freelancerId String
  freelancer  User        @relation(fields: [freelancerId], references: [id])
  bidAmount   Float
  coverLetter  String
  status      String      @default("PENDING")
  createdAt   DateTime    @default(now())
}

```

```

}

model Contract {
  id          String      @id @default(cuid())
  jobId       String
  job         Job          @relation(fields: [jobId], references: [id])
  clientId    String
  client      User          @relation("ClientContracts", fields: [clientId], references: [id])
  freelancerId String
  freelancer  User          @relation("FreelancerContracts", fields: [freelancerId], references: [id])
  amount      Float
  status      ContractStatus @default(ACTIVE)
  milestones  Milestone[]
  createdAt   DateTime      @default(now())
}

model Milestone {
  id          String      @id @default(cuid())
  contractId  String
  contract    Contract @relation(fields: [contractId], references: [id])
  title       String
  amount      Float
  isFunded    Boolean      @default(false)
  isApproved  Boolean      @default(false)
}

```

5. PHASE-BY-PHASE IMPLEMENTATION ROADMAP

Phase 1: Project Initialization & Auth Foundation

Weeks 1 - 2

- Initialize Next.js 15 App Router project with TypeScript, Tailwind CSS, and Shadcn UI.
- Configure Prisma with PostgreSQL database schema and initial migrations.
- Implement role-based Authentication (Client, Freelancer, Admin profile onboarding).
- Build profile creation wizards with custom skill tags, hourly rate selectors, and portfolio uploaders.

Phase 2: Core Marketplace Engine (Jobs & Proposals)

Weeks 3 - 4

- Create Client Job Posting engine (Wizard with Fixed/Hourly budget, required skills, and attachments).
- Build Freelancer Job Search feed with multi-filter facets (Category, Budget range, Skill tags).
- Build Proposal submission workflow with milestone definition and pricing calculations.
- Implement Client Proposal Review dashboard with shortlist, decline, and interview actions.

Phase 3: Realtime Engine & Contract/Escrow Pipeline

Weeks 5 - 6

- Integrate WebSocket/Socket.io messaging engine for real-time chat between Client and Freelancer.
- Implement Stripe Connect integration for Escrow creation: deposit funds into hold status upon contract award.
- Build Milestone Submission & Approval workflow (Deliver work -> Review -> Release funds to Freelancer wallet).

Phase 4: 3D UI Visual Layer Integration

Weeks 7 - 8

- Integrate `@react-three/fiber` and `@spline-tool/react-spline` into Next.js App Router structure.
- Build 3D Landing Page Hero scene with dynamic floating skill cards and global talent nodes.
- Render 3D Dashboard canvas representing milestone progress with animated lighting effects.
- Implement fallback loading states and frame-budget optimizations for 60fps performance across mobile and desktop.

Phase 5: Super Admin Dashboard, Dispute Handling & Launch

Weeks 9 - 10

- Build Admin Control Center: user account moderation, platform metrics (GMV, platform fees revenue), and dispute resolution system.
- Perform security audit: rate limiting API routes, sanitizing proposal input, and securing S3 file access policies.
- Deploy production infrastructure on Vercel (Frontend/Edge functions) and Supabase/Neon PostgreSQL.